

MindEase

A plain-English guide to the whole project

This document explains what MindEase is, how it is built, what technology it uses, how its database and APIs work, and what every file in the project does — written in simple language so a non-expert can follow it.

Part	Language / Tool
Backend (the server)	Python 3.11 + FastAPI
Database	MongoDB (via Motor async driver)
Frontend (the website)	React + TypeScript + Vite
Styling	Tailwind CSS + shadcn/ui components
AI - face emotion	OpenCV DNN: YuNet + HSEmotion (ONNX)
AI - chat	Groq API (LLM) with a local fallback
AI - text sentiment	VADER (rule-based)

Generated on 2026-09-07. File: MindEase_Project_Guide.pdf

1. What MindEase is

MindEase is a mental-wellness web app. A person signs in, then picks a path:

- "With Community & AI": the app looks at a webcam photo, guesses the mood on the face, then opens an AI chat for support, and a community area to connect with others.
- "Solo Expression": the person writes freely, the app reads the text and suggests exercises, quotes and videos, then asks for feedback.

So the core ideas are: check how you feel (by face or by writing), get gentle support (AI chat + curated content), and optionally connect with a small community.

2. The big picture (how the parts fit together)

MindEase has three moving parts that talk over the internet:

Part	What it is	What it does
Frontend	A React website the user sees in the browser	Shows screens, takes input (typing, webcam), calls the backend, shows the answers.
Backend	A FastAPI server (Python)	Checks who you are, runs the logic, talks to the database and the AI, returns clean JSON.
Database	MongoDB	Stores users, mood history, chat messages, community posts, connections, feedback.

The backend is organised in clear layers so each file has one job:

```
Route  ->  Controller  ->  Service  ->  Model  ->  Database
(URL)      (thin glue)   (real work) (shape a (MongoDB)
                        document)
```

- Route: defines the URL (for example POST /api/auth/login) and who may call it.
- Controller: a tiny pass-through that calls the right service function.
- Service: where the actual work happens (checking a password, saving a mood, calling the AI).
- Model: a small helper that builds the dictionary that gets saved to MongoDB.
- Schema: describes the exact shape of the data coming in and going out, and rejects bad input.

3. Technology used, and why

Technology	Where	Why it was chosen (simple reason)
FastAPI	Backend framework	Fast, modern Python; automatic input checking and API docs.
Uvicorn	Backend web server	Runs the FastAPI app and handles incoming connections.
MongoDB	Database	Flexible 'document' storage; easy to store varied wellness data.
Motor	DB driver	Lets the Python server talk to MongoDB without blocking.
Pydantic	Data validation	Describes and checks the shape of every request and response.
python-jose	Tokens	Creates and reads the JWT login tokens.
passlib + bcrypt	Passwords	Turns passwords into a safe one-way hash before storing.
slowapi	Rate limiting	Blocks a client that sends too many requests too fast.
OpenCV (cv2)	Vision	Reads the webcam image, finds the face, runs the emotion model.
ONNX models	AI weights	Pre-trained face detector (YuNet) and emotion model (HSEmotion).
Groq SDK	Chat AI	Calls a large language model in the cloud for chat replies.
vaderSentiment	Text mood	Rule-based English sentiment scoring, no model download.
React 18	Frontend UI	Component-based screens; large ecosystem.
TypeScript	Frontend language	JavaScript with types; catches mistakes before running.
Vite	Frontend build tool	Instant dev server and fast production build.
Tailwind CSS	Styling	Utility classes for consistent, quick styling.
shadcn/ui + Radix	UI components	Accessible buttons, dialogs, tabs, sliders, etc.
React Router	Navigation	Moves between screens without full page reloads.
TanStack Query	Data fetching	Loads and caches server data on some screens.
framer-motion	Animation	Smooth fades and transitions.

4. How a request travels (three examples)

Example A - Signing in

- The Login screen sends email + password to POST /api/auth/login.
- auth_service looks up the user by email, and checks the password against the stored bcrypt hash.
- If correct, it creates two tokens: an access token (short-lived) and a refresh token.
- The browser saves the access token and sends it on every later request in the 'Authorization' header.

Example B - Emotion (face) check

- EmotionAnalysis screen turns on the webcam, captures one frame, and sends it as base64 to POST /api/emotion/detect.
- detector.py decodes the image, uses YuNet to find exactly one face (0 faces or 2+ faces = friendly error).
- The face is cropped and passed to the HSEmotion model, which returns one of: happiness, sadness, anger, fear, neutrality (surprise and disgust are mapped to the closest of these).
- The result is saved in the 'emotions' collection and returned to the screen with a confidence score.

Example C - Chatting with MindEase AI

- ChatAI screen sends the message (and an optional 'mode' like breathing or pep_talk) to POST /api/chat/send.
- First the backend checks for crisis words. If found, it returns a caring safety message with a helpline and never sends it to the AI.
- Otherwise it builds a short prompt that tells the AI to answer in 2-4 plain sentences, and calls Groq.
- If the AI is unreachable, a local rule-based reply is used instead.
- The message and reply are saved so the conversation can be reloaded later.

5. The database (MongoDB)

MongoDB stores 'documents' (like JSON objects) grouped into 'collections' (like tables). MindEase creates these collections. Every document also has an auto '_id' and a 'created_at' time.

users

Field	Meaning
name, email	Who the person is (email is unique).
password_hash	The bcrypt hash of the password (never the real password).
token_version	A counter; bumping it on logout makes old tokens stop working.
emotional_history	List of emotion-record ids for this user.
feedback_history	List of feedback ids.
communities	Names of communities the user joined.
connections	Ids of accepted peer connections.

emotions (and reasons)

Field	Meaning
user_id	Which user this reading belongs to.
emotion	The detected mood word.
confidence	How sure the model is (0 to 1).
description	A short supportive sentence for that mood.
reason	Optional note the user typed about why they feel this way.

The 'reasons' collection keeps the same note linked to the emotion id, for history.

chat_messages

Field	Meaning
user_id	Whose conversation this is.
user_message	What the person typed.
ai_reply	What MindEase answered.
provider_used	'groq' if the cloud AI answered, 'local' if the offline fallback did.

community_posts

Field	Meaning
user_id / text	Author and the post text ('seed' for starter posts).
mood	Mood guessed from the text (used for the coloured badge).
ai_reply	A supportive reply the AI adds to each post.
likes / liked_by	Like count and the list of users who liked it.

Field	Meaning
comments / raw_comments	Plain comment texts, and full comment objects with author + id.
community_name	Which community the post belongs to (optional).

communities

name (unique), and members (list of user ids).

connection_requests

Field	Meaning
from_user_id / to_user_id	Who asked to connect with whom.
status	'pending', 'accepted' or 'rejected'.

When accepted, each user's 'connections' list is updated so they can direct-message.

direct_messages

conversation_id (the two user ids sorted and joined), from_user_id, to_user_id, body. Used by the real-time chat between connected peers.

solo_analyses

Field	Meaning
user_id / session_id	Owner and a session grouping id.
text	What the user wrote on the 'Page of Expression'.
emotion / confidence / model_used	The detected feeling and how it was found.
suggestions	Exercises, quotes, tips and videos tailored to that feeling.

feedback

user_id, flow_type ('with_flow' or 'without_flow'), content_type (quotes/exercises/videos), message, rating (1 to 5).

Indexes (making lookups fast)

On startup the backend creates indexes so common lookups are quick: unique email on users; user_id on emotions, reasons, chat_messages, feedback; paired indexes on connection_requests and direct_messages; a time index on community_posts; unique name on communities.

6. Security and login, in simple words

Protection	What it means here
Password hashing (bcrypt)	Passwords are turned into an irreversible scramble before saving. Even someone with the database cannot read them.
JWT access token	After login the user gets a signed token. It is sent on every request to prove identity. It expires after a set time.
Refresh token	A longer-lived token meant to get a new access token (issued now; a refresh endpoint is a planned addition).
token_version	Stored on the user. Logout increases it, which instantly invalidates any token that was already handed out.
Rate limiting (slowapi)	Limits how many requests one client can make per minute, to slow down brute-force and abuse.
CORS rules	Only the app's own web addresses are allowed to call the API from a browser.
Input validation (Pydantic)	Every request body is checked against a strict shape; bad or oversized input is rejected with a clear 422 error.
Safe errors	The server logs the full error internally but returns a short, non-revealing message to the user.
Secrets in .env	Database URL, JWT secret and API keys live in an untracked .env file, not in the code.

Every protected endpoint runs 'get_current_user' first: it opens the token, checks the signature and expiry, loads the user, and checks token_version still matches. If anything fails, the request is refused with 401.

7. The APIs (endpoints)

All paths start with /api. 'Auth' means a valid login token is required.

Authentication - /api/auth

Method + Path	Auth	What it does
POST /signup	no	Create an account; returns access + refresh tokens.
POST /login	no	Check email + password; returns tokens.
POST /logout	yes	Bumps token_version so existing tokens stop working.
GET /me	yes	Returns the current user's profile (no password).

Emotion (face) - /api/emotion

Method + Path	Auth	What it does
POST /detect	yes	Takes a webcam image (base64 or file); returns the detected mood + confidence; saves it.
GET /history	yes	Last 50 mood readings for this user.
POST /attach-reason	yes	Adds or updates the 'why' note on a past reading.

Chat - /api/chat

Method + Path	Auth	What it does
POST /send	yes	Send a message (+ optional mode); returns the AI reply. Handles crisis words safely.
GET /history	yes	Reload the saved conversation.
POST /clear	yes	Delete this user's chat history.

Solo text analysis - /api/solo

Method + Path	Auth	What it does
POST /analyze	yes	Reads written text, detects the feeling, returns tailored suggestions.

Community - /api/community

Method + Path	Auth	What it does
POST /join	yes	Join (or create) a community by name.
GET /feed	yes	Recent posts, trending posts, suggested users.
POST /post	yes	Create a post; the AI adds a supportive reply and a mood badge.
POST /like	yes	Like or unlike a post.
POST /comment	yes	Add a comment to a post.
GET /details/{name}	yes	Community name, member count, recent posts.

Connections and direct messages - `/api/connection`

Method + Path	Auth	What it does
POST <code>/request</code>	yes	Ask to connect with a user (by email or id).
POST <code>/accept, /reject</code>	yes	Respond to an incoming request.
GET <code>/state</code>	yes	Incoming, outgoing and accepted connections.
GET <code>/users/search?q=</code>	yes	Find users by name or email.
GET <code>/messages/{peer_id}</code>	yes	Load the direct-message history with one connection.

Feedback - `/api/feedback`

Method + Path	Auth	What it does
POST <code>/submit</code>	yes	Save a star rating + comment for the session.

Real-time - WebSocket `/ws/social`

A live connection (opened with the token in the URL) used for instant direct messages and 'your connections changed' notifications, so the Community screen updates without refreshing.

Health - `/health`

GET `/health` returns 'ok' when the server is up. Add `?ready=true` to also check the database.

8. The AI features explained

Face emotion detection

- Step 1 - Find the face: YuNet (a small, accurate face-detector model) locates the face in the photo. If it finds none, or more than one, the user gets a clear message instead of a wrong guess.
- Step 2 - Read the expression: the face is cropped and given to HSEmotion, an EfficientNet model trained on real-world face photos (AffectNet). It returns the most likely emotion with a score.
- Step 3 - Map to app words: the 8 model emotions are mapped to the 7 the app uses (surprise and contempt become 'neutrality'/'anger').
- Fallbacks: if YuNet is missing it uses OpenCV's older Haar detector; if HSEmotion is missing it uses the older FER+ model. Model files download automatically on first use and are cached.

The chatbot

- Style: the prompt tells the AI to reply in 2-4 short sentences, plain everyday words, warm tone, acknowledge the feeling then give one small step or one gentle question.
- Provider: it calls Groq (a fast LLM service). The model name is set in providers.py and can be overridden with the GROQ_MODEL environment variable.
- Tools / modes: buttons send a 'mode' (breathing, grounding, journal_prompt, reframe, pep_talk) that steers the reply to a specific short exercise or message.
- Crisis safety: messages with self-harm language are caught before the AI and answered with a caring message pointing to a crisis line. This is not a substitute for professional help.
- Offline fallback: if no AI provider answers, a local rule-based generator gives a short, on-topic reply (it also understands the modes).
- Memory: the last few messages are included as context, and the whole conversation is saved so it can be reloaded.

Text sentiment (VADER)

For written text (Solo Expression, community posts, chat context) the app uses VADER, a rule-based English sentiment tool. It needs no download and maps the text to one of the seven feelings.

9. Frontend walkthrough (the screens)

Screen (route)	What the user does there
Login (/)	Sign in or sign up. On success the token is saved and the app goes to Home.
Home (/home)	Welcome page; 'Begin Your Journey' -> path selection.
PathSelection (/path-selection)	Choose 'With Community & AI' or 'Solo Expression'.
EmotionAnalysis (/emotion-analysis)	Use the webcam (or upload a photo) to detect mood, then optionally add a note.
ChatAI (/chat-ai)	Chat with MindEase AI. Suggestion chips, tool buttons (Breathe / Grounding / Journal prompt / Pep talk), a breathing timer, copy, and clear.
Community (/community)	Join the group, read/write posts, like and comment, search users, send connection requests, and direct-message connections in real time.
Expression (/expression)	Write freely; the app analyses the text and shows tailored exercises, quotes, tips and videos.
ContentSelection (/content-selection)	Pick which kinds of content matter to you (quotes, videos, exercises, etc.).
Feedback (/feedback)	Rate the session 1-5 stars and leave a comment.
ThankYou (/thank-you)	Closing screen; also has 'Log Out'.
NotFound (*)	Shown for any unknown URL.

Shared frontend helpers

- lib/api.ts - one place that calls every backend endpoint and attaches the login token.
- lib/camera.ts - safe webcam access with clear messages when it is blocked or missing.
- lib/authFlow.ts - remembers the chosen path and content picks (in the browser) for the feedback step.
- lib/utils.ts - a tiny helper to combine CSS class names.
- hooks/useRequireAuth.ts - sends the user back to Login if there is no token.
- hooks/useSocialWebSocket.ts - manages the live connection for community messages.
- components/FloatingOrbs, PageTransition, NavLink - background blobs, fade animation, active links.
- components/ui/* - the shadcn/ui building blocks (buttons, dialogs, tabs, sliders, toasts...).

10. Backend files, one line each

app/config

File	Job
settings.py	Reads .env into one settings object (DB URL, JWT secret, limits, model choice).
database.py	Opens the MongoDB connection and creates all indexes on startup.
logging.py	Sets up console logging format and level.

app (entry)

File	Job
main.py	Creates the FastAPI app, adds CORS + rate-limit + error handlers, includes every route file, seeds the community feed, exposes /health.

app/routes (the URLs)

File	Job
auth_routes.py	signup / login / logout / me.
emotion_routes.py	detect / history / attach-reason (reads JSON or multipart upload).
chat_routes.py	send / history / clear.
solo_routes.py	analyze.
community_routes.py	join / feed / post / like / comment / details.
connection_routes.py	request / accept / reject / state / user search / messages.
feedback_routes.py	submit.
social_ws.py	the /ws/social WebSocket for live messages.

app/controllers (thin glue)

auth, emotion, chat, solo, community, connection, feedback - each just forwards the request to its matching service function.

app/services (the real work)

File	Job
auth_service.py	Sign up, log in, log out, return profile; hashing and token creation.
emotion_service.py	Runs the detector, saves the reading, updates history, handles the reason note.
chat_service.py	Builds recent-message context, calls the AI engine, saves the exchange, history + clear.
ai_engine.py	Decides the prompt style, detects crisis words, adds mode instructions, calls providers, falls back to local.
chat_fallback.py	Offline reply generator: short, warm, randomised, mode-aware.
intent.py	Quick guess of what the user wants (help, question, motivation, thanks...).

File	Job
local_nlp.py	VADER sentiment -> one of the seven feelings, with a confidence.
solo_service.py	Classifies written text and builds the exercises/quotes/tips/videos pack.
community_service.py	Join or create a community; community details.
community_feed_service.py	Create posts (with AI reply + mood), feed, likes, comments, seed starter posts.
social_service.py	User search, connection requests/accept/reject, connection state, direct messages.
feedback_service.py	Save a feedback document and link it to the user.

app/ai

providers.py - tries Groq, then HuggingFace, then Ollama, then GPT4All; returns the first reply or an offline message. (The chatbot's model name is set here.)

app/emotion_model

detector.py - the whole face pipeline: decode image, YuNet/Haar face detection, HSEmotion/FER+ emotion model, map to app words, plus automatic model download.

app/models (document builders)

user_model, emotion_model, reason_model, chat_model, solo_model, feedback_model, community_model, connection_model - each returns the exact dict that gets saved to MongoDB.

app/schemas (shapes and validation)

auth, emotion, chat, solo, community, community_feed, feedback, user - Pydantic classes that define and check every request and response body.

app/middleware

File	Job
auth_middleware.py	get_current_user: verify the token, load the user, check token_version.
rate_limit.py	Sets up the slowapi limiter used across the app.

app/utils

File	Job
common.py	Small shared helpers: current time, ObjectId conversion, id serialisation.
security.py	hash_password, verify_password, create access/refresh tokens.
ws_auth.py	Reads the token from a WebSocket connection and returns the user.

app/realtime

social_manager.py - keeps track of which users have a live WebSocket open, and pushes messages to them.

other backend files

File	Job
requirements.txt	The list of Python packages to install.
run_backend.sh	Helper script to free port 8000 and start the server.
.env / .env.example	Real secrets / a safe template of the settings.
check_community_data.py, clear_community_data.py	Small dev scripts to inspect or reset community data.

11. Frontend files, one line each

Setup / config

File	Job
index.html	The single HTML page React mounts into.
src/main.tsx	Starts React and renders <App/>.
src/App.tsx	Defines every route (URL -> screen) and wraps the app in providers.
src/index.css	Design tokens (colours, fonts), the glass-card look, buttons, dark-mode vars.
tailwind.config.ts	Tailwind theme: the sage/teal/peach palette, fonts, animations.
vite.config.ts	Dev server on port 8080 and a proxy so /api and /ws go to the backend.
vitest.config.ts, eslint.config.js, tsconfig*.json	Test, lint and TypeScript settings.

src/pages (screens)

Login, Home, PathSelection, EmotionAnalysis, ChatAI, Community, Expression, ContentSelection, Feedback, ThankYou, NotFound - described in section 9.

src/lib

File	Job
api.ts	Every backend call in one file; adds the Authorization header; formats errors.
camera.ts	Cross-browser webcam access with friendly failure messages.
authFlow.ts	Stores the chosen journey and content picks in the browser.
utils.ts	cn() - merge CSS class names.

src/hooks

File	Job
useRequireAuth.ts	Redirect to Login when there is no token.
useSocialWebSocket.ts	Open/maintain the live socket; expose sendDm().
use-mobile.tsx, use-toast.ts	Detect small screens; show toast notifications.

src/components

File	Job
FloatingOrbs.tsx	The soft blurred background blobs.
PageTransition.tsx	Fade/slide wrapper for screens.
NavLink.tsx	A link that knows when it is the active page.
ui/*	shadcn/ui primitives (button, dialog, tabs, slider, sonner toasts, etc.).

12. How to run it

Prerequisites

- Python 3.11
- Node.js 18+
- MongoDB running locally (or a cloud URL)

Backend

```
cd backend
py -3.11 -m venv .venv311
.venv311\Scripts\activate
pip install -r requirements.txt
copy .env.example .env      (then edit JWT_SECRET_KEY, MONGODB_URL, GROQ_API_KEY)
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev      (opens http://localhost:8080)
```

The frontend talks to the backend through Vite's proxy, so no extra configuration is needed in development. Health check: <http://localhost:8000/health>

13. What changed recently (this round of work)

Face emotion detection - accuracy fix

- Problem: the old model (FER+, trained on tiny 48x48 grayscale images) leaned heavily toward 'neutral' and got most expressions wrong.
- Fix: switched to HSEmotion (EfficientNet-B0, trained on real-world AffectNet photos). It runs in the same OpenCV runtime - no new Python package.
- Also added: YuNet face detection, a 'more than one face' guard, cleaner image preprocessing, and automatic model download.
- Result on a 7-expression test set: from about 3/7 to 7/7 correct.
- Only one file changed: backend/app/emotion_model/detector.py.

Chatbot - rebuild

- Found the AI was completely down: the configured model 'llama-3.1-8b-instant' was removed by Groq (every call returned 404). Switched to 'openai/gpt-oss-20b'.
- Answers are now short and plain: 2-4 sentences, everyday words, acknowledge the feeling then one small step or gentle question.
- New interface: header navigation buttons, tappable suggestion chips, a tools row (Breathe / Grounding / Journal prompt / Pep talk), an interactive breathing timer, copy button, clear-conversation, and a saved-history reload.
- New safety: crisis words are caught before the AI and answered with a helpline message.
- New endpoints: GET /api/chat/history and POST /api/chat/clear; /api/chat/send now accepts a 'mode'.
- Files changed: the chat route/controller/service/schema, ai_engine.py, chat_fallback.py, providers.py (the model-name fix), and on the frontend ChatAI.tsx and api.ts.

14. Known limitations and possible next steps

- The emotion vocabulary has only 7 words, so 'surprise' and 'disgust' are folded into 'neutrality'/'anger'. Adding them properly is a small schema + UI change.
- There is no `/api/auth/refresh` endpoint yet, so when the access token expires the user logs in again. Token rotation and a 'my sessions' view are natural additions.
- No email verification or password reset yet.
- The chatbot depends on the Groq key and model being valid; the model name is now overridable with the `GROQ_MODEL` environment variable.
- Community 'members' data on the sidebar is partly illustrative sample data.
- Model files (YuNet, HSEmotion, FER+) are downloaded on first use; the first request after a fresh install is slower.

End of document. MindEase - plain-English project guide.